Supporto all'internazionalizzazione
Culture locali e formattatori

Corso di Laurea in Ingegneria Informatica

Anno accademico 2025/2026

Prof. ENRICO DENTI
Dipartimento di Informatica – Scienza e Ingegneria (DISI)

Premessa: il pattern factory

UNA FABBRICA PER OGGETTI

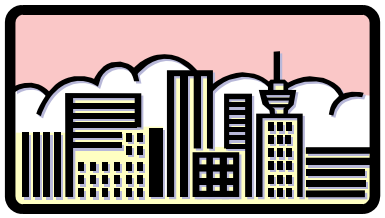
- In molte situazioni pratiche è *inopportuno* lasciar costruire direttamente gli oggetti all'utente
  - Rischio che non rispetti i vincoli
  - Oggetti complessi hanno costruttori complessi, con molti argomenti
  - Oggetti complessi richiedono *una competenza e conoscenza fuori dalla portata degli utenti* per essere costruiti bene
- IDEA: far costruire gli oggetti a una *fabbrica* apposita
  - <u>Incapsula la conoscenza necessaria</u>
  - Nasconde i dettagli del procedimento di costruzione

IL PATTERN "FACTORY" (fabbrica)

- È un altro famosissimo *design pattern*
  - non è il primo che incontriamo: nel codice fiscale avevamo già visto (un caso semplice di) Façade..
- La *fabbrica* nasconde agli utenti la costruzione di oggetti
  - l'utente non può costruirli da solo: deve *farseli fare dalla fabbrica*
  - solo la fabbrica sa come costruirli
  - tecnicamente: la fabbrica espone una o più *funzioni statiche (factory method)* che *incapsulano e nascondono la new*
  - nomi tipici dei metodi factory:
    - in passato, `get**`
    - oggi più spesso `of` (talora, `valueOf`)

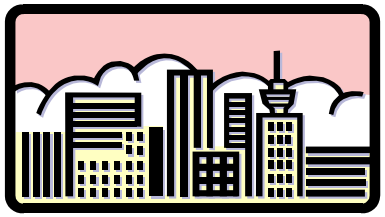
PERCHÉ UNA FABBRICA?

- Perché costruire un oggetto complesso spesso richiede *molto di più di una "semplice new"*
- Perché una fabbrica può applicare *decisioni strategiche* su cosa e come costruire
  - può decidere di costruire *oggetti fatti in un modo o in un altro* in base ai parametri passati o alla situazione contingente
  - può *legare la costruzione alla disponibilità di risorse,* pagamenti, o condizioni al contorno
  - può perfino *"costruire per finta" un oggetto,* prendendolo in realtà da un insieme prestabilito di oggetti già pronti
- Perché potrebbe essere utile *costruire tipi diversi (ma compatibili..) di oggetti* senza che il cliente lo debba sapere ☺



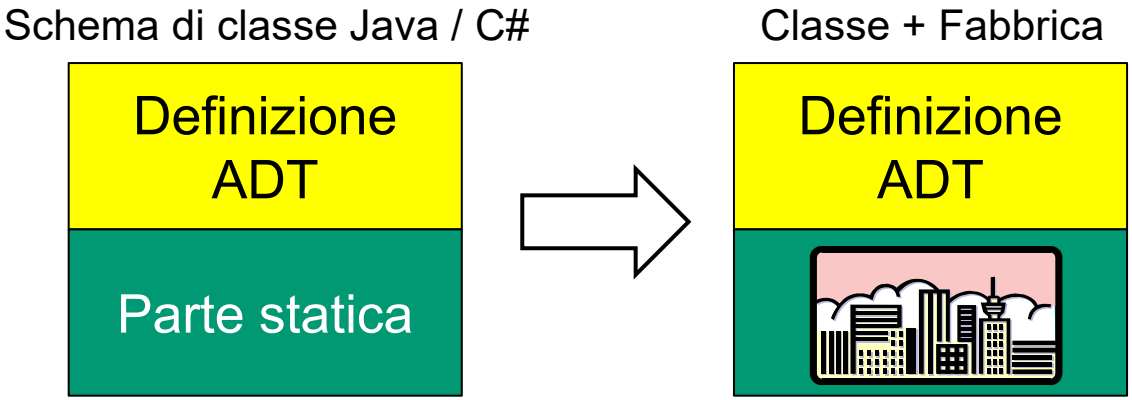
COME È FATTA UNA FABBRICA?

- Esternamente, una fabbrica si presenta come un insieme di funzioni statiche (*factory methods*) che incapsulano la new
  - nome tipico: **`of`** (o a volte **`valueOf`** o più anticamente **`getXXX`**)
  - calibrano la costruzione in base agli argomenti ricevuti
  - restituiscono un oggetto creato *secondo le politiche aziendali*
- Se presente, la fabbrica è tipicamente la sola entità autorizzata a costruire quel certo tipo di oggetti
  - i costruttori di tali oggetti non sono più pubblici
  - la fabbrica e l'oggetto vengono *progettate insieme* (stessa classe o stesso package) per lavorare in tandem.



SCHEMA TIPICO

- Per tenere insieme l'oggetto e la sua fabbrica, si sfrutta il fatto che in una classe possa esserci una parte statica:
  - la fabbrica è incorporata nella stessa classe che definisce il tipo
  - i costruttori *non sono pubblici*
  - sostituiti da una o più *funzioni statiche* che mascherano e incapsulano tutti i dettagli di costruzione



FABBRICHE EVERYWHERE!

- Troveremo applicato questo schema ovunque!
- Ad esempio, nelle classi che gestiscono formati, internazionalizzazione, orari, date, etc.
  - cultura locale: classe **`Locale`**
  - formattatori per numeri: classe **`NumberFormat`**
  - tutto il package **`java.time`**
  - tutti i formattatori per date e orari: package **`java.time.format`**
  - …e altri in futuro

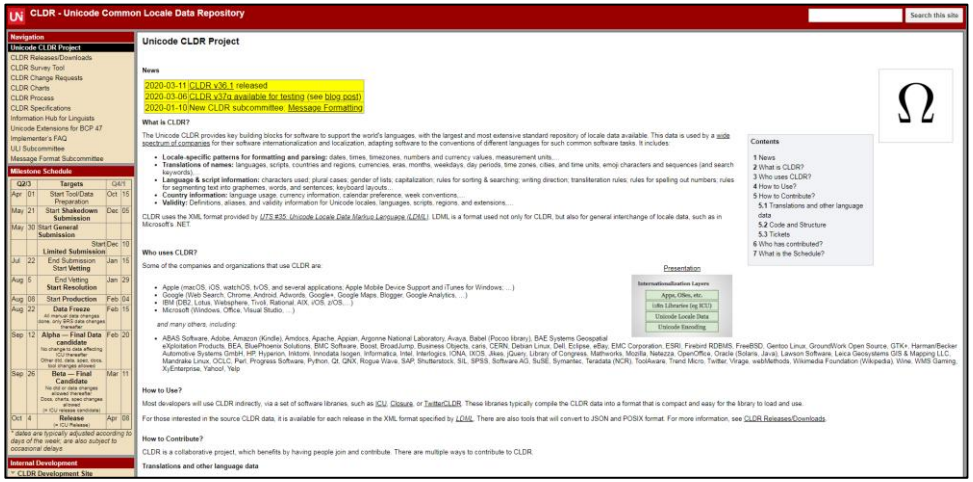
Il concetto di cultura locale

CONVENZIONI DI FORMATTAZIONE E CULTURE LOCALI

- Le convenzioni per numeri, valute, percentuali, date, orari non sono universali: *ogni paese, ogni cultura ha le proprie*
  - separatori ammessi per le migliaia e le parti decimali dei numeri
  - simboli di valuta, posizione del simbolo (prima/dopo il valore)
  - date: nomi dei giorni della settimana, nomi dei mesi, ordine con cui compaiono gli elementi (gg/mm/aa, mm/gg/aa…), separatore, etc.
  - orari: su 12 o 24 ore, mezzogiorno/mezzanotte, separatore, etc.
  - … e molto altro
- Appositi *formattatori* (o metodi di formattazione) provvedono a operare in base alla *cultura locale* richiesta
  - insiemi di regole per *stampa* (output) e *parsing* (input) di stringhe

DATABASE DI CULTURE E FORMATI

- Appositi *database* raccolgono tali convenzioni
  - Java adotta, da Java 9, il *database internazionale Unicode CLDR (Common Locale Data Repository)*
    - fino a Java 8 si usava un *proprio database interno,* chiamato «JRE», simile ma *non identico* a CLDR



    - in particolare, occhio alla posizione del simbolo € nei prezzi italiani..!
  - C# adotta invece il *Microsoft Language Code ID* (MS-LCID)
  - Scala e Kotlin si appoggiano alla soluzione Java sottostante
    - almeno per ora (Kotlin ha in sviluppo una sua libreria)

LA CULTURA LOCALE IN Java

- In Java, la classe **`java.util.Locale`** rappresenta appunto il concetto di *cultura locale*
  - insiemi di regole per la *generazione di stringhe* (per la stampa)…
  - …e anche, a rovescio, per il *parsing* di numeri, valute, date, orari
- Una *cultura locale* è composta da *lingua* e *paese*
  - MOTIVO: una lingua è tipicamente parlata in più paesi e regioni, che spesso adottano *convenzioni diverse*
  - la *lingua* è espressa da una sigla *minuscola di due lettere*
  - il *paese* è espresso da una sigla *MAIUSCOLA di due lettere*
  - ad es. l'italiano non è parlato solo in Italia, ma anche in Svizzera: la valuta e le regole di formattazione dei numeri *non sono le stesse* → la cultura locale «Italia» è **`it_IT`** , la «Svizzera italiana» è **`it_CH`**
  - idem per l'inglese: **`en_GB`**, **`en_US`**, **`en_CANADA`**,…

LA CULTURA LOCALE IN Java

- Alcune costanti "scorciatoia" abbreviano i casi più frequenti:

Modifier and Type	Field	Description
static Locale	CANADA	Useful constant for country.
static Locale	CANADA_FRENCH	Useful constant for country.
static Locale	CHINA	Useful constant for country.
static Locale	CHINESE	Useful constant for language.
static Locale	ENGLISH	Useful constant for language.
static Locale	FRANCE	Useful constant for country.
static Locale	FRENCH	Useful constant for language.
static Locale	GERMAN	Useful constant for language.
static Locale	GERMANY	Useful constant for country.
static Locale	ITALIAN	Useful constant for language.
static Locale	ITALY	Useful constant for country.
static Locale	JAPAN	Useful constant for country.
static Locale	JAPANESE	Useful constant for language.
static Locale	KOREA	Useful constant for country.
static Locale	KOREAN	Useful constant for language.
static Locale	PRC	Useful constant for country.
static char	PRIVATE_USE_EXTENSION	The key for the private use extension ('x').
static Locale	ROOT	Useful constant for the root locale.
static Locale	SIMPLIFIED_CHINESE	Useful constant for language.
static Locale	TAIWAN	Useful constant for country.
static Locale	TRADITIONAL_CHINESE	Useful constant for language.
static Locale	UK	Useful constant for country.
static char	UNICODE_LOCALE_EXTENSION	The key for Unicode locale extension ('u').
static Locale	US	Useful constant for country.

LA CULTURA LOCALE IN Java

- In tutti gli altri casi bisogna procurarsi il giusto **`Locale`**
  - fino a Java 18, tramite opportuni costruttori

Constructors

Constructor	Description
`Locale(String language)`	Construct a locale from a language code.
`Locale(String language, String country)`	Construct a locale from language and country.
`Locale(String language, String country, String variant)`	Construct a locale from language, country and variant.

  - da Java 19, tramite i nuovi *factory methods* **`of`** e **`forLanguageTag`**

Constructors

Constructor	Description
`Locale(String language)`	**Deprecated.** Locale constructors have been deprecated.
`Locale(String language,`	
`Locale(String language,`	

Costruttori deprecati perché non facevano alcun controllo sintattico sull'input

Static Methods

`static Locale`	`of(String language)`
`static Locale`	`of(String language, String country)`
`static Locale`	`of(String language, String country, String variant)`
`static Locale`	`forLanguageTag(String languageTag)`

LA CULTURA LOCALE IN Java

- Se non si specifica nulla, i formattatori adottano il *default,* ottenibile dalla funzione statica*:*
  - `Locale.getDefault`
- Per conoscere tutte le culture locali disponibili basta invocare la funzione statica :
  - `Locale.getAvailableLocales`

LA CULTURA LOCALE IN Java: ESEMPI

```
jshell> Locale.getDefault();
$47 ==> it_IT
```

Java

It_IT è la sigla della cultura locale in uso

```
jshell> Locale italy = Locale.ITALY;
italy ==> it_IT

jshell> italy.getCountry();
$41 ==> "IT"

jshell> italy.getLanguage();
$42 ==> "it"
```

Java

Locale ITALY predefinito

I metodi get* recuperano le singole proprietà

```
jshell> var loc=Locale.forLanguageTag("it-CH")
loc ==> it_CH

jshell> loc.getLanguage();
$44 ==> "it"

jshell> loc.getCountry();
$46 ==> "CH"
```

Java

Per la Svizzera italiana non esiste un Locale predefinito → occorre costruirsi l'oggetto

Java: formattatori numerici

(Scala, Kotlin)

FORMATTATORI NUMERICI

- Ci sono due grandi famiglie di formattatori:
  - formattatori *numerici* → ne parliamo ora
  - formattatori per *date e orari* → ne parleremo più avanti
- I formattatori numerici (classe **`java.text.NumberFormat`**) si occupano di:
  - formattazione di numeri
  - formattazione di percentuali
  - formattazione di prezzi/valute
- Costruzione con *pattern factory* (metodi **`getXXX`**) distinti per *numeri, percentuali, valute*
- Uso: **`formatter.format(`*`valore`*`)`**

FORMATTATORI NUMERICI

- I formattatori numerici si ottengono tutti dalla classe **`java.text.NumberFormat`**, *che funge anche da factory*
- Costruzione tramite i tre metodi **`get**(…)`** distinti per *numeri*, *percentuali* e *valute*

Java

```
double x = 43.12345678, y = 0.7, z = 13456.78;

NumberFormat fN = NumberFormat.getNumberInstance();

NumberFormat fP = NumberFormat.getPercentInstance();

NumberFormat fV = NumberFormat.getCurrencyInstance();
```

Formattatore numeri

Formattatore percentuali

Formattatore valute

ESEMPIO: FORMATTAZIONE DI NUMERI

Java

```
double x = 43.12345678, y = 0.7, z = 13456.78;
NumberFormat fN = NumberFormat.getNumberInstance();
fN.setMaximumFractionDigits(2);
System.out.println( fN.format(x) );
System.out.println( fN.format(y) );
System.out.println( fN.format(z) );
```

FACTORY

Configurazione

Output:

```
43,12
0,7
13.456,78
```

MAX due cifre decimali, come richiesto

Il default è **`Locale.ITALY`**: usa il punto come separatore delle migliaia e la virgola come separatore decimale

ESEMPIO: FORMATTAZIONE DI NUMERI

Java

```
double x = 43.12345678, y = 0.7, z = 13456.78;
NumberFormat fN = NumberFormat.getNumberInstance(
                                    Locale.CANADA);
fN.setMaximumFractionDigits(2);
System.out.println( fN.format(x) );
System.out.println( fN.format(y) );
System.out.println( fN.format(z) );
```

Cambiando cultura locale..

Output:

```
43.12
0.7
13,456.78
```

.. cambia il risultato: il Canada ha convenzioni opposte!

ESEMPIO: FORMATTAZIONE DI NUMERI

Java

```
double x = 43.12345678, y = 0.7, z = 13456.78;
NumberFormat fN = NumberFormat.getNumberInstance(
                                    Locale.CANADA_FRENCH);
fN.setMaximumFractionDigits(2);
System.out.println( fN.format(x) );
System.out.println( fN.format(y) );
System.out.println( fN.format(z) );
```

.. e cambiando ancora, si replica!

Output:

```
43,12
0,7
13 456,78
```

Però, la parte francofona adotta convenzioni ancora diverse!

Notare lo spazio (hard) come separatore delle migliaia!

Non è lo spazio standard: è un carattere diverso

Infatti, in Jshell si vede un carattere spurio:

```
123á456,78
```

FORMATTAZIONE DI PERCENTUALI

Java

```
double p = 0.4312, q = 0.7, r = 1.2345678;
NumberFormat fP = NumberFormat.getPercentInstance(…);
fP.setMaximumFractionDigits(2);
System.out.println( fP.format(p) );
System.out.println( fP.format(q) );
System.out.println( fP.format(r) );
```

FACTORY

Formattatore percentuali configurato su due cifre decimali

Default (Italia):

```
43,12%
70%
123,46%
```

Canada French:

```
43,12 %
70 %
123,46 %
```

UK:

```
43.12%
70%
123.46%
```

Notare lo spazio (hard) prima del simbolo di percentuale !

FORMATTAZIONE DI VALUTE

- La formattazione di *valori valuta* è analoga…
- .. ma nel caso ITALIA costituisce un punto delicato, perché *il passaggio al database CLDR ha cambiato la convenzione sulla posizione del simbolo di valuta, €*
  - fino a Java 8, il simbolo Euro era posizionato *prima* dell'importo
  - da Java 9, il simbolo Euro viene posizionato *dopo* l'importo, perché CLDR adotta una diversa convenzione al riguardo
- Rompendo la retrocompatibilità, ciò ha causato problemi
  - formattazioni diverse eseguendo in Java9+ codice scritto per Java8
  - peggio: *il parsing di stringhe col simbolo € davanti all'importo fallisce in Java9+ usando il formattatore standard italiano* → ATTENZIONE

CLDR (cldr.unicode.org)

www.unicode.org/cldr/charts/latest/by_type/numbers.number_formatting_patterns.html

CLDR Charts — Home | Site Map | Searc

CLDR v48.0

By-Type Chart: Numbers:Number Formatting Patterns

2025-10-25

Index

Core Data:	Alphabetic Information | Main Exemplars | Punctuation Exemplars | Index Exemplars | Numbering Systems
Locale Display Names:	Locale Name Patterns | Languages (A-D) | Languages (E-J) | Languages (K-N) | Languages (O-S) | Languages (T-Z) | Scripts | Geographic Regions | Territories (North America) | Territories (South America) | Territories (Africa) | Territories (Europe) | Territories (Asia) | Territories (Oceania) | Locale Variants | Keys
Date & Time:	Fields | Relative | Gregorian | Gregorian Y
Timezones:	Timezone Display Patterns | North Amer … a | Unknown Region | Overrides
Numbers:	Symbols | Number Formatting Patterns
Currencies:	North America (C) | South America (C) | Central Asia (C) | Eastern Asia (C) | Sou
Units:	Measurement Systems | Duration | Grap … Electrical and Frequency | Weather | Dig
Characters:	Category | Smileys & Emotion | People … tion | Math Symbols | Other Symbols | Flags |
Miscellaneous:	Displaying Lists | Minimal Pairs | Person Name Formats

LEGENDA

- Il carattere jolly ¤ rappresenta il generico simbolo di valuta
- Il carattere jolly # rappresenta una generica cifra (che può mancare)
- il carattere 0 rappresenta la generica cifra (che deve essere presente)
- Il carattere virgola rappresenta il generico separatore di gruppo
- Il carattere punto rappresenta il generico separatore decimale

Standard Patterns

standard-decimal	**English: ‹#,##0.###›**
#,##,##0.###	·as· ·bn· ·brx· ·ccp· ·dv· ·dz· ·en_IN· ·gu· ·hi· ·kok· ·kok_Latn· ·kxv· ·kxv_Deva· ·kxv_Orya· ·kxv_Telu· ·ml· ·mr· ·ne· ·or· ·pa· ·sa· ·ta· ·te· ·xnr·
#,##0.###	**·ta_MY· ·ta_SG··all·others·**
#,#0.###	·tok·
0.######	
standard-currency	
#,##,##0.00¤	
#,##0.00 ¤	
#,##0.00 ¤	
#,##0.00 ¤;-#,##0.00 ¤	
#,##0.00¤	·agq· ·bez· ·dje· ·kab· ·khq· ·km· ·ksb· ·lg· ·lu· ·luo· ·oc· ·rn· ·rwk· ·sbp· ·seh· ·ses· ·shi· ·shi_Latn· ·twq· ·zgh·
¤ #,##,##0.00	·as· ·brx· ·dv· ·ne·
¤ #,##0.00	**·de_AT· ·de_LI· ·en_AT· ·en_MV· ·es_AR· ·es_CO· ·es_PE· ·es_UY· ·fa_AF· ·ms_Arab_BN· ·ms_BN· ·ta_MY· ·ta_SG··all·others·**
¤ #,##0.00	·bqi· ·fa·
¤ #,##0.00;¤ -#,##0.00	·blo· ·en_NL· ·es_PY· ·ie· ·nl·
¤ #,##0.00;¤ #,##0.00-	·fy·
¤ #,##0.00;¤-#,##0.00	·de_CH· ·en_CH· ·it_CH·
¤ 0.00	·en_US_POSIX·

#,##0.00 ¤

·ab· ·an· ·asa· ·ast· ·az· ·az_Cyrl· ·bas· ·be· ·bg· ·br· ·bs· ·bs_Cyrl· ·ca· ·ce· ·co· ·cs· ·cu· ·cv· ·da· ·de· ·dsb· ·dua· ·dyo· ·el· ·en_150· ·eo· ·es· ·et· ·eu· ·ewo· ·ff· ·fi· ·fo· ·fr· ·gl· ·gsw· ·hr· ·hsb· ·hu· ·hy· ·is· ·it· ·ka· ·kea· ·kk· ·ksf· ·ksh· ·ku· ·ky· ·lb· ·lij· ·ln· ·lt· ·lv· ·mk· ·my· ·nds· ·nmg· ·pl· ·prg· ·pt_PT· ·rif· ·rm· ·ro· ·ru· ·sah· ·sc· ·se· ·sk· ·sl· ·smn· ·sq· ·sr· ·sr_Latn· ·sv· ·szl· ·tg· ·tk· ·tpi· ·tt· ·tzm· ·uk· ·uz· ·uz_Arab· ·uz_Cyrl· ·vi· ·vmw· ·xog· ·yav·

¤ #,##0.00;¤ #,##0.00-	·fy·
¤ #,##0.00;¤-#,##0.00	·de_CH· ·en_CH· ·it_CH·
¤ 0.00	·en_US_POSIX·

FORMATTAZIONE DI VALUTE

```
double x = 1243.5678;
NumberFormat fV = NumberFormat.getCurrencyInstance(…);
System.out.println( fV.format(x) );
```

Java

FACTORY

Formattatore valute

Default (Italia):

```
1.243,57 €
```

Canada:

```
$1,243.57
```

UK:

```
£1,243.57
```

Italia Java 8:

```
€ 1.243,57
```

Canada French:

```
1 243,57 $
```

Svizzera italiana:

```
SFr. 1’243.57
```

NB: nelle valute non serve specificare il numero di cifre decimali: fanno già parte della convenzione, *con relativi arrotondamenti*

Notare lo spazio (hard)

Notare lo spazio (hard) e l’apice particolare

PARSING DI STRINGHE NUMERICHE

- Sebbene nascano per «formattare» (in output), i formattatori supportano anche *l'operazione duale*, detta *parsing* (in input)
- Il metodo **`parse`** converte una stringa numerica
  - *correttamente formattata* secondo le regole di una *cultura locale*
  - rispettivamente come valore numerico, percentuale o valuta

  in un valore numerico di classe **`Number`**
  - facilmente estraibile come **`double`**, **`float`** o **`int`** a piacere
- MA il suo uso richiede la gestione degli errori (eccezioni), poiché se la stringa è scorretta il formattatore "si arrabbia"
  - possiamo però fare qualche esperimento interattivo con ***jshell***, in quanto la console interattiva *gestisce da sola gli errori* ☺

PARSING DI STRINGHE NUMERICHE: ESEMPI

Java

```
NumberFormat fV =
                NumberFormat.getCurrencyInstance(Locale.US);
System.out.println(fV.parse("$123.56"));
```

Formattatore USA

```
jshell> System.out.println(fv.parse("$123.56"))
123.56
```

Il metodo **parse** restituisce un oggetto **Number**, che si può poi convertire in **double**, **float** o **int** con gli opportune arrotondamenti:

```
jshell> Number n = fV.parse("$123.456789987654321")
n ==> 123.45678998765432

jshell> double d = n.doubleValue()
d ==> 123.45678998765432

jshell> float f = n.floatValue()
f ==> 123.45679
```

PARSING DI STRINGHE NUMERICHE: ESEMPI

Java

```
NumberFormat fP =
            NumberFormat.getPercentInstance(Locale.US);
fP.setMinimumFractionDigits(2);
System.out.println(fP.parse("72.35%"));
```

Formattatore USA

```
jshell> fP.parse("72.35%")
$12 ==> 0.7234999999999999
```

Da notare che *se invece del locale USA si usasse il locale ITALY*, il comportamento sarebbe diverso e *rischioso*:

```
jshell> fP.parse("72.35%")
$14 ==> 72.35

jshell> fP.parse("72,35%")
$15 ==> 0.7234999999999999
```

Formattatore IT OCCHIO!

Formattatore IT ok

PERICOLO! La prima stringa viene interpretata *solo parzialmente,* di fatto *ignorando il simbolo di % (ORRORE!),* restituendo un risultato assurdo!

PARSING DI STRINGHE NUMERICHE: ESEMPI

Java

Ora proviamo con il locale Canada francese:
(notare la virgola e l'uso dello spazio prima del simbolo di percentuale)

```
jshell> NumberFormat fP = NumberFormat.getPercentInstance(Locale.CANADA_FRENCH)
fP.setMinimumFractionDigits(2)

jshell> fP.format(0.7235)
$12 ==> "72,35 %"
```

Facendo il parsing senza spazio, BOOM!

No, non è uno spazio normale!

```
jshell> fP.parse("72,35 %")
|   java.text.ParseException thrown: Unparseable number: "72,35 %"
|         at NumberFormat.parse (NumberFormat.java:431)
|         at (#13:1)

jshell> fP.parse("72,35%")
|   java.text.ParseException thrown: Unparseable number: "72,35%"
|         at NumberFormat.parse (NumberFormat.java:431)
|         at (#14:1)

jshell> fP.parse("72,35\u00A0%")
$15 ==> 0.7234999999999999
```

OK solo con l' hard space

Perché funzioni, nella stringa servono la virgola e il *non breakable space*

CI PENSEREMO…

Java

- La gestione efficace dei vari metodi **`parse`**, con annessi e connessi, sarà affrontata a tempo debito
  - ora come ora, tentando di usare **`parse`** in un vostro programma *otterrete errore di compilazione* perché in Java la gestione di quegli errori è <u>obbligatoria</u> e voi ancora non sapete farla!
  - ergo, se volete fare (piccoli) esperimenti, usate **`jshell`** …
  - .. ma <u>evitando il simbolo €</u> perché non è ASCII (scegliete $ o £)
  - …o, meglio, abbiate pazienza e aspettate qualche settimana ☺
- Per ora, ricordate che:
  - formattare valori numerici in stringa è *facile* e *sicuro*
  - l'inverso invece non è *immediato* e può dar luogo a *errori*

FORMATTATORI PERSONALIZZATI

- Non vi piacciono/bastano i formattatori standard?
  - vorreste *il simbolo di € davanti* invece che in fondo nelle valute?
- In Java potete farvi il vostro *formattatore personalizzato*
  - per numeri e valute, tramite la classe **`DecimalFormat`**, il cui costruttore accetta una *stringa di formato* che specifica il pattern richiesto

```
DecimalFormat formatter = new DecimalFormat(pattern);
```

  - ovviamente, a patto di studiarsi i (molti) simboli del ***`pattern`***… ☺

FORMATTATORI PERSONALIZZATI
Esempio: valute con € davanti

- Da Java 9, l'adozione del database CLDR ha spostato il simbolo dell'Euro *dopo* il valore nel **`Locale.ITALY`**
  - ciò può essere sgradevole esteticamente, ma soprattutto creare *problemi in fase di parsing* di prezzi col simbolo € davanti
- Si può ovviare costruendosi un formattatore personalizzato
  - si sfrutta la classe specifica **`java.text.DecimalFormat`**
  - NOTA: qui si usa la costruzione diretta, *senza factory*, perché occorre poter specificare *intenzionalmente* tutti i dettagli!

```
DecimalFormat formatter = new DecimalFormat(pattern);
```

  - QUI si specifica come pattern di formato "**`¤ #,##0.##`**"
  - il simbolo di valuta è rappresentato dal carattere jolly "**`¤`**", la generica cifra numerica dal carattere jolly "**`#`**"

FORMATTATORI PERSONALIZZATI
Esempio: valute con € davanti

- Più precisamente, il pattern "`¤ #,##0.##`" specifica:
  - uno spazio dopo il simbolo di valuta (carattere jolly "`¤`")
  - separatori delle migliaia ogni tre cifre (carattere "`,`" posizionato in questo caso tre posti a sinistra del punto decimale "`.`")
  - uno zero davanti per i numeri che abbiano intera nulla (carattere "`0`")
  - due cifre decimali a destra del punto decimale (caratteri "`##`")
- ATTENZIONE: sebbene il pattern usi i simboli anglosassoni
  - "`,`" come separatore delle migliaia e "`.`" come separatore decimale

  il formattatore li sostituirà con quelli della cultura locale
  - quindi, per l'Italia il separatore delle migliaia diventerà "`.`" e il separatore della parte decimale diverrà "`,`")

FORMATTATORI PERSONALIZZATI
Esempio: valute con € davanti

- Java: formattatore personalizzato:

```
DecimalFormat f = new DecimalFormat("¤ #,##0.##");
```

- Esempi di formattazioni:

```
System.out.println(f.format(1234.567));
System.out.println(f.format(-1234.567));
System.out.println(f.format(0.567));
System.out.println(f.format(12345678.91234));
```

```
€ 1.234,57
-€ 1.234,57
€ 0,57
€ 12.345.678,91
```

Arrotondamento automatico alla seconda cifra decimale ☺

Però, i valori negativi con "-" davanti non sono splendidi… ☹

FORMATTATORI PERSONALIZZATI
Esempio: valute con € davanti

- Si può perfezionare dando una doppia specifica:
  - la prima valida per valori positivi
  - la seconda valida per valori negativi

```
var f = new DecimalFormat("¤  #,##0.##;¤ -#,##0.##");
```

Specifica per valori negativi

Magari due spazi per i positivi, così da garantire l'incolon-namento anche col «-»

- Esempi di formattazioni:

```
System.out.println(f.format(1234.567));
System.out.println(f.format(-1234.567));
System.out.println(f.format(0.567));
System.out.println(f.format(12345678.91234));
```

```
€  1.234,57
€ -1.234,57
€  0,57
€  12.345.678,91
```

Ora è tutto molto più carino ☺

Java

C#: formattazione di numeri

FORMATTAZIONI IN C#

- Come in Java, anche qui si distingue fra:
  - formattazione di *numeri* → ne parliamo ora
  - formattazione di *date e orari* → ne parleremo più avanti
- Come in Java, anche qui esiste il concetto di *cultura locale*, espresso dalla classe `System.Globalization.CultureInfo`
  - ESEMPIO: `CultureInfo itLocale =`
    `new System.Globalization.CultureInfo("it-IT");`
- A differenza di Java, però, spesso *non è necessario* creare il formattatore esplicitamente: si usa la *string interpolation*
  - ricorda i formati della `printf` del C
  - si genera la stringa formattata indicando l'appropriata *specifica di formato* nel metodo che genera la stringa (es. `WriteLine`)

CULTURE LOCALI IN C#

- Le culture locali si distinguono in *neutre* («it») e *specifiche* («it_IT»)
  - neutre: solo la lingua, prescindendo da uno specifico paese
  - specifiche: coppia lingua + Paese

ESEMPIO

```
Console.WriteLine("{0,-20}{1,4}", "#All cultures:",
        CultureInfo.GetCultures(CultureTypes.AllCultures).Length);

Console.WriteLine("{0,-20}{1,4}", "#Neutral cultures:",
        CultureInfo.GetCultures(CultureTypes.NeutralCultures).Length);

Console.WriteLine("{0,-20}{1,4}", "#Specific cultures:",
        CultureInfo.GetCultures(CultureTypes.SpecificCultures).Length);
```

C#

```
#All cultures:       354
#Neutral cultures:   144
#Specific cultures:  210
```

.NET Framework 4
C# 5.0

```
#All cultures:       733
#Neutral cultures:   223
#Specific cultures:  510
```

.NET Framework 5
C# 6+

CULTURE LOCALI IN C#

- Una tabella per elencarle tutte..?

```
Console.OutputEncoding = System.Text.Encoding.UTF8;

Console.WriteLine("{0,-15}{0,-5}{0,-45}{0,-40}",
                  "Culture", "ISO", "Display name", "English Name");

foreach (CultureInfo ci in CultureInfo.GetCultures(CultureTypes.AllCultures)) {
            Console.Write("{0,-15}",     ci.Name);
            Console.Write("{0,-5}",      ci.TwoLetterISOLanguageName);
            Console.Write("{0,-45}",     ci.DisplayName);
            Console.WriteLine("{0,-40}", ci.EnglishName);
};
```

Utile per vedere anche nomi con simboli «strani»

C#

```
Culture         CultureCulture                                    Culture
                iv                                  Invariant Language (Invariant Country)
af              af   af                             Afrikaans
af-NA           af   Afrikaans (Namibië)            Afrikaans (Namibia)
af-ZA           af   af (ZA)                        Afrikaans (South Africa)
...
it              it   it                                       Italian
it-CH           it   it (CH)                                  Italian (Switzerland)
it-IT           it   it (IT)                                  Italian (Italy)
it-SM           it   italiano (San Marino)                    Italian (San Marino)
it-VA           it   italiano (Città del Vaticano)            Italian (Vatican City)
```

FORMATTAZIONE DI NUMERI IN C#

- La formattazione di numeri avviene tramite *specifiche di formato* direttamente nel metodo di stampa **`WriteLine`**
  - tali specifiche seguono lo schema **`{id:specifier}`** dove ***`id`*** è l'indice numerico dell'argomento (1,2,3,…), elencato dopo la stringa stessa, e ***`specifier`*** è un carattere-specificatore (opzionalmente seguito dalla *larghezza del campo,* se richiesta)
  - il carattere **`N`** indica formattazione numerica
  - il carattere **`c`** indica formattazione di valuta (currency)
  - il carattere **`p`** indica formattazione percentuale

ESEMPIO BASE

```
Console.WriteLine("x={0:N}, y={1:c}, z={2:p}", x,y,z);
```

Numeric Currency Percentage

FORMATTAZIONE DI NUMERI: ESEMPI

C#

```
double x = 43.12345678, y = 0.7, z = 13456.78, w = 13_456.78;
double u = 0.4312, v = 0.7, r = 1.2345678;

CultureInfo itIT = new CultureInfo("it-IT");
CultureInfo itCH = new CultureInfo("it-CH");
CultureInfo enGB = new CultureInfo("en-GB");
CultureInfo enCA = new CultureInfo("en-CA");
CultureInfo frCA = new CultureInfo("fr-CA");
CultureInfo deDE = new CultureInfo("de-DE");

CultureInfo.DefaultThreadCurrentCulture = enUK;

Console.WriteLine($"Current culture: {CultureInfo.CurrentCulture.Name}");
```

```
Current culture: en-GB
x = 43.12345678, y = 0.7,
z = 13456.78, w = 13,456.780,
w = £13,456.78
u = 43.12%, v = 70.00%, r = 123.46%
```

String interpolation with prefix (C#6.0)

String interpolation: argument #3 (z) as Currency

```
Console.WriteLine("x = {0}, y = {1}, z = {2}, w = {3:N}, w = {3:c}" , x,y,z,w);
```

String interpolation: argument #0 (x)

String interpolation: argument #3 (z) as Number

```
Console.WriteLine("u = {0:p2}, v = {1:p2}, r = {2:p2}" , u,v,r);
```

String interpolation: argument #0 (u) as Percentage with 2 decimal digits

FORMATTAZIONE DI NUMERI: ESEMPI

C#

```
Current culture:
x = 43.12345678, y = 0.7, z = 13456.78, w = 13,456.78, w = ¤13,456.78
p = 43.12 %, q = 70.00 %, r = 123.46 %

Current culture: en-GB
x = 43.12345678, y = 0.7, z = 13456.78, w = 13,456.780, w = £13,456.78
p = 43.12%, q = 70.00%, r = 123.46%

Current culture: en-CA
x = 43.12345678, y = 0.7, z = 13456.78, w = 13,456.780, w = $13,456.78
p = 43.12%, q = 70.00%, r = 123.46%

Current culture: fr-CA
x = 43,12345678, y = 0,7, z = 13456,78, w = 13 456,780, w = 13 456,78 $
p = 43,12 %, q = 70,00 %, r = 123,46 %

Current culture: it-IT
x = 43,12345678, y = 0,7, z = 13456,78, w = 13.456,780, w = 13.456,78 €
p = 43,12%, q = 70,00%, r = 123,46%

Current culture: it-CH
x = 43.12345678, y = 0.7, z = 13456.78, w = 13'456.780, w = CHF 13'456.78
p = 43.12%, q = 70.00%, r = 123.46%

Current culture: de-DE
x = 43,12345678, y = 0,7, z = 13.456,780, w = 13.456,780, w = 13.456,78 €
p = 43,12 %, q = 70,00 %, r = 123,46 %
```

Default: indefinita

Canada (en/fr)

Italia vs. Svizzera it.

Germania: spazio prima del simbolo di %

PARSING DI STRINGHE NUMERICHE

- In C# , il parsing viene gestito dai metodi statici **`TryParse`**
  - analizzano la stringa e ne determinano *in modo smart* il formato
  - l'output è un argomento *passato per indirizzo (keyword* **`out`***):* il risultato restituito, invece, è un **`Bool`** che riporta successo/fallimento
  - ci sono anche i metodi (non statici) **`Parse`** e **`ParseExact`** che però lanciano errore se la stringa non rispetta il formato previsto
- Ci sono fondamentalmente due versioni di **`TryParse`**
  - quella a due argomenti, che usa la cultura predefinita
  - quella a quattro argomenti, che consente di specificare la cultura

ESEMPIO

```
double d;
if (double.TryParse(numString, out d)) …
if (double.TryParse(numString, culture, style, out d)) …
```

Passaggio per indirizzo

C#

PARSING DI NUMERI: ESEMPI E CONTROESEMPI

C#

```
using System;
using System.Globalization;

public class Program
{
    public static void Main() {
        CultureInfo[] cultures = { new CultureInfo("it-IT") , new CultureInfo("en-US")} ;
        string[] numStrings = { "12€", "12 €", "12,34 €", "€ 12", "€12", "€ 12,54", "€ 12.21", "13", "$ 13", "13.66$", "13,77$" };

        foreach (CultureInfo culture in cultures) {
            Console.WriteLine("Cultura: {0}", culture);
            foreach (string numString in numStrings){
                double d;
                if( double.TryParse(numString, NumberStyles.Currency, culture, out d))
                    Console.WriteLine("La stringa {0} è stata convertita in {1} secondo la cultura {2}", numString, d, culture);
                else
                    Console.WriteLine("ERRORE: la stringa {0} non è stata riconosciuta nella cultura {1}", numString, culture);
            }
        }
    }

}
```

```
Cultura: it-IT
La stringa 12€ è stata convertita in 12 secondo la cultura it-IT
La stringa 12 € è stata convertita in 12 secondo la cultura it-IT
La stringa 12,34 € è stata convertita in 12.34 secondo la cultura it-IT
La stringa € 12 è stata convertita in 12 secondo la cultura it-IT
La stringa €12 è stata convertita in 12 secondo la cultura it-IT
La stringa € 12,54 è stata convertita in 12.54 secondo la cultura it-IT
La stringa € 12.21 è stata convertita in 1221 secondo la cultura it-IT
La stringa 13 è stata convertita in 13 secondo la cultura it-IT
ERRORE: la stringa $ 13 non è stata riconosciuta nella cultura it-IT
ERRORE: la stringa 13.66$ non è stata riconosciuta nella cultura it-IT
ERRORE: la stringa 13,77$ non è stata riconosciuta nella cultura it-IT
Cultura: en-US
ERRORE: la stringa 12€ non è stata riconosciuta nella cultura en-US
ERRORE: la stringa 12 € non è stata riconosciuta nella cultura en-US
ERRORE: la stringa 12,34 € non è stata riconosciuta nella cultura en-US
ERRORE: la stringa € 12 non è stata riconosciuta nella cultura en-US
ERRORE: la stringa €12 non è stata riconosciuta nella cultura en-US
ERRORE: la stringa € 12,54 non è stata riconosciuta nella cultura en-US
ERRORE: la stringa € 12.21 non è stata riconosciuta nella cultura en-US
La stringa 13 è stata convertita in 13 secondo la cultura en-US
La stringa $ 13 è stata convertita in 13 secondo la cultura en-US
La stringa 13.66$ è stata convertita in 13.66 secondo la cultura en-US
La stringa 13,77$ è stata convertita in 1377 secondo la cultura en-US
```

OCCHIO a quel che succede usando il separatore decimale sbagliato..!